



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий
Кафедра Математического обеспечения и стандартизации информационных
технологий

Отчет по итоговому проекту №2
по дисциплине «Технологии разработки программных приложений»

Тема работы: «Разработка проекта и системы сборки»

Выполнили:

Студенты группы ИНБО-20-23

Акулова А.П.

Боргачев Т.М.

Свирин А.О.

Проверил:

Ассистент кафедры МОСИТ,

Петрова А.А.

Москва 2025

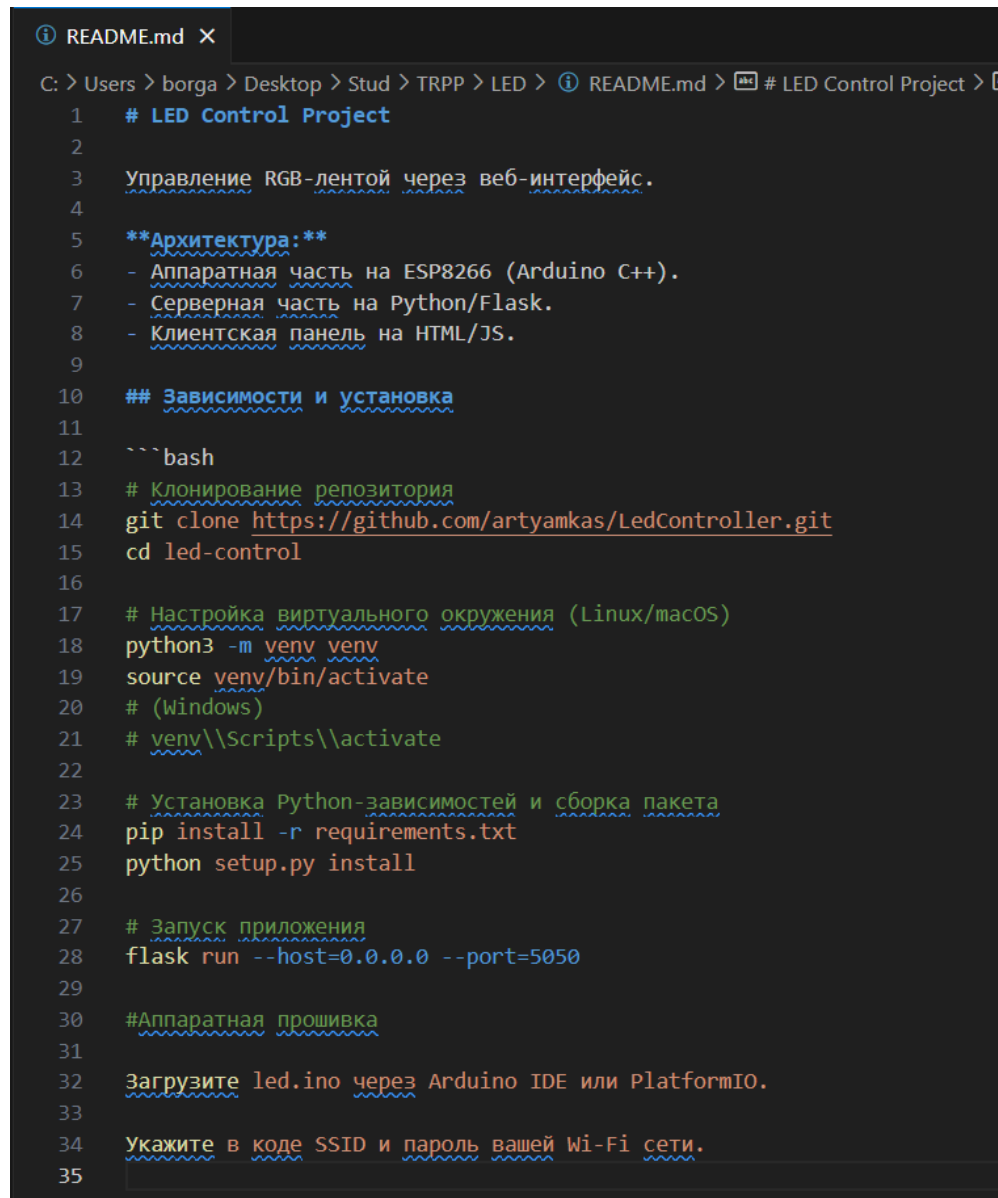
СОДЕРЖАНИЕ

1 РАЗРАБОТКА ПРОЕКТА И СИСТЕМЫ СБОРКИ	3
1.1 Файл README.md.....	3
1.2 Зависимости Python проекта	3
1.3 Серверная часть проекта	4
1.4 Аппаратная часть проекта	12
1.5 Сборка документации проекта.....	16

1 РАЗРАБОТКА ПРОЕКТА И СИСТЕМЫ СБОРКИ

1.1 Файл README.md

Написанный файл с описанием проекта и инструкцией для запуска представлен на рис. 1.



```
1  # LED Control Project
2
3  Управление RGB-лентой через веб-интерфейс.
4
5  **Архитектура:**
6  - Аппаратная часть на ESP8266 (Arduino C++).
7  - Серверная часть на Python/Flask.
8  - Клиентская панель на HTML/JS.
9
10 ## Зависимости и установка
11
12 ```bash
13 # Клонирование репозитория
14 git clone https://github.com/artyamkas/LedController.git
15 cd led-control
16
17 # Настройка виртуального окружения (Linux/macOS)
18 python3 -m venv venv
19 source venv/bin/activate
20 # (Windows)
21 # venv\Scripts\activate
22
23 # Установка Python-зависимостей и сборка пакета
24 pip install -r requirements.txt
25 python setup.py install
26
27 # Запуск приложения
28 flask run --host=0.0.0.0 --port=5050
29
30 #Аппаратная прошивка
31
32 Загрузите led.ino через Arduino IDE или PlatformIO.
33
34 Укажите в коде SSID и пароль вашей Wi-Fi сети.
35
```

Рисунок 1 – Файл README.md

1.2 Зависимости Python проекта

Python-часть проекта представляет собой сервер, написанный на flask. Таким образом файл с зависимостями будет выглядеть как представлено на рис. 2.

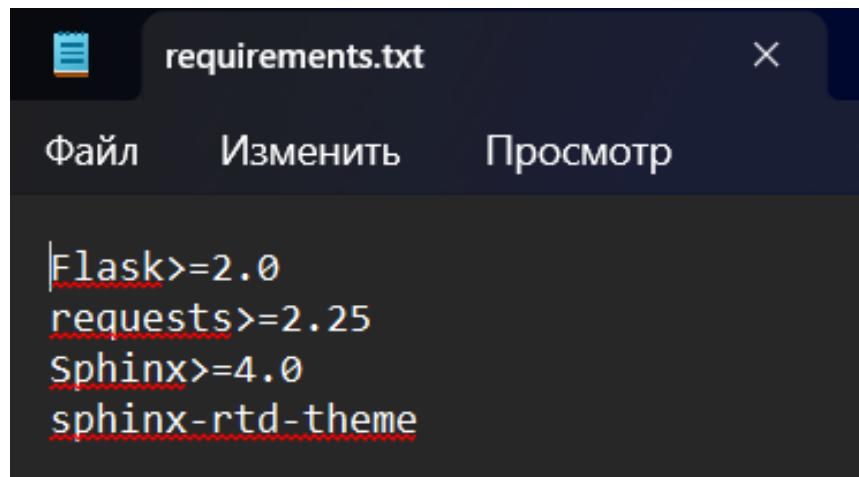


Рисунок 2 – Файл requirements.txt

1.3 Серверная часть проекта

Код для файла setup.py представлен в листинге 1.

Листинг 1 – Код для setup.py

```
from setuptools import setup, find_packages
from sphinx.setup_command import BuildDoc

setup(
    name="led_control",
    version="0.1.0",
    packages=find_packages(),
    include_package_data=True,
    install_requires=[
        "Flask>=2.0",
        "requests>=2.25",
    ],
    entry_points={
        'console_scripts': [
            'led-control=app:main'
        ]
    },
    setup_requires=["sphinx", "sphinx-rtd-theme"],
    cmdclass={
        'build_sphinx': BuildDoc
    },
    command_options={
        'build_sphinx': {
            'project': ('setup.py', 'LED Control'),
            'version': ('setup.py', '0.1.0'),
            'source_dir': ('setup.py', 'docs')
        }
    }
)
```

Код приложения app.py представлен в листинге 2.

Листинг 2 – Код для app.py

```
import socket
from flask import Flask, render_template, request, redirect, jsonify, session
import requests
import os
app = Flask(__name__)
app.secret_key = os.urandom(24)
def get_local_ip():
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    s.connect(("8.8.8.8", 80))
    local_ip = s.getsockname()[0]
    s.close()
    return local_ip
DEFAULT_ESP_IP = 'http://192.168.1.100'
def get_esp_ip():
    return session.get('esp_ip', DEFAULT_ESP_IP)
@app.route('/')
def index():
    return render_template('index.html', current_ip=get_esp_ip())
@app.route('/update_ip', methods=['POST'])
def update_ip():
    new_ip = request.form.get('ip')
    if new_ip and new_ip.startswith(('http://', 'https://')):
        session['esp_ip'] = new_ip
        return jsonify({'status': 'success', 'ip': new_ip})
    return jsonify({'status': 'error'}), 400
@app.route('/power/<state>')
def power_control(state):
    esp_ip = get_esp_ip()
    if state == 'on':
        requests.get(f'{esp_ip}/on')
    else:
        requests.get(f'{esp_ip}/off')
    return redirect('/')
@app.route('/color')
def color():
    esp_ip = get_esp_ip()
    r = request.args.get('r')
    g = request.args.get('g')
    b = request.args.get('b')
    if all([r, g, b]):
        requests.get(f'{esp_ip}/color?r={r}&g={g}&b={b}')
    return redirect('/')
@app.route('/mode', methods=['POST'])
def set_mode():
    esp_ip = get_esp_ip()
    mode = request.form.get('mode', '0')
    requests.get(f'{esp_ip}/mode?m={mode}')
    return ('', 200)
if __name__ == '__main__':
    app.run(host=get_local_ip(), port=5050)
```

HTML-файл для обертки сайта представлен в листинге 3.

Листинг 3 – index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>LED Control Panel</title>
  <meta name="current-ip" content="{ { current_ip } }">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <link rel="stylesheet" href="{ { url_for('static', filename='style.css') } }">
  <!-- Подключаем библиотеку Pickr -->
  <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/@simonwep/pickr/dist/themes/classic.min.css">
</head>
<body>
  <header class="header">
    <h1 class="title">Управление LED-лентой</h1>
    <div class="status" id="status">Подключение...</div>
  </header>
  <div class="control-grid">
    <div class="power-control">
      <div class="power-status">
        <span>Состояние: </span>
        <span id="powerStateText">Выключен</span>
        <span id="powerIndicator" class="indicator off"></span>
      </div>

      <div class="power-buttons">
        <a id="powerOnLink"><button id="powerOnBtn" class="power-
btn">Включить</button></a>
        <a id="powerOffLink"><button id="powerOffBtn" class="power-btn"
disabled>Выключить</button></a>
      </div>
    </div>
    <div class="color-control">
      <h3>Выбор цвета</h3>
      <div class="current-color-section">
        <div class="color-label">Текущий цвет:</div>
        <div id="colorPreview" class="color-preview"></div>
      </div>
      <div id="colorPicker" class="pickr"></div>
      <button id="applyColorBtn" class="color-btn">Применить цвет</button>
    </div>
    <div class="mode-control">
      <h3>Выбор режима</h3>
      <div class="current-mode">
        <span>Текущий режим: </span>
        <span id="currentModeText">Статичный</span>
      </div>
      <div class="mode-buttons">
        <button data-mode="0" class="mode-btn active">Статичный</button>
```

```

        <button data-mode="1" class="mode-btn">Дыхание</button>
        <button data-mode="2" class="mode-btn">Радуга</button>
        <button data-mode="3" class="mode-btn">Музыка</button>
    </div>
</div>
</div>
<div id="ipModal" class="modal">
    <div class="modal-content">
        <span class="close">&times;</span>
        <h2>Настройки подключения</h2>
        <label for="ipInput">IP-адрес контроллера:</label>
        <div class="ip-input-container">
            <input type="text" id="ipInput" value="http://192.168.1.100">
        </div>
        <button id="saveIpBtn" class="save-btn">Сохранить</button>
    </div>
</div>
<div id="connectionAlert" class="connection-alert">
    <div class="alert-content">
        <p>Не удалось подключиться к устройству</p>
        <p class="alert-hint">Нажмите на статус для настройки подключения</p>
    </div>
</div>
<script
src="https://cdn.jsdelivr.net/npm/@simonwep/pickr/dist/pickr.min.js"></script>
<script>
    document.addEventListener('DOMContentLoaded', function() {
        const statusEl = document.getElementById('status');
        const ipModal = document.getElementById('ipModal');
        const ipInput = document.getElementById('ipInput');
        const saveIpBtn = document.getElementById('saveIpBtn');
        const closeModal = document.querySelector('.close');
        const powerOnBtn = document.getElementById('powerOnBtn');
        const powerOffBtn = document.getElementById('powerOffBtn');
        const powerIndicator = document.getElementById('powerIndicator');
        const powerStateText = document.getElementById('powerStateText');
        const alertBox = document.getElementById('connectionAlert');

        // Загружаем состояние из localStorage (по умолчанию false - выключено)
        let isDeviceOn = localStorage.getItem('deviceState') === 'true';

        // Функция обновления состояния
        function updatePowerState() {
            if (isDeviceOn) {
                powerIndicator.className = 'indicator on';
                powerStateText.textContent = 'Включен';
                powerOnBtn.disabled = true;
                powerOffBtn.disabled = false;
            } else {
                powerIndicator.className = 'indicator off';
                powerStateText.textContent = 'Выключен';
            }
        }
    });

```

```

        powerOnBtn.disabled = false;
        powerOffBtn.disabled = true;
    }
}
// Обработчики кнопок
powerOnBtn.addEventListener('click', async () => {
    const response = await fetch(`/power/on`);
    if (response.ok) {
        isDeviceOn = true;
        localStorage.setItem('deviceState', 'true');
        updatePowerState();
    }
});

powerOffBtn.addEventListener('click', async () => {
    const response = await fetch(`/power/off`);
    if (response.ok) {
        isDeviceOn = false;
        localStorage.setItem('deviceState', 'false');
        updatePowerState();
    }
});

updatePowerState();

// Загружаем IP из сервера при загрузке
let controllerURL = 'http://192.168.1.100'; // дефолтное значение

// Функция загрузки текущего IP с сервера
async function loadCurrentIP() {
    try {
        const response = await fetch('/');
        const text = await response.text();
        const parser = new DOMParser();
        const doc = parser.parseFromString(text, 'text/html');
        const ipMeta = doc.querySelector('meta[name="current-ip"]');
        if (ipMeta) {
            controllerURL = ipMeta.content;
            ipInput.value = controllerURL;
        }
    } catch (e) {
        console.error('Error loading current IP:', e);
    }
}

// Проверка подключения
async function checkConnection() {
    statusEl.textContent = "Проверка подключения...";
    statusEl.style.color = "#666";

    try {
        await new Promise(resolve => setTimeout(resolve, 500));
        const res = await fetch(`${controllerURL}/`, {

```



```

        method: "GET",
        mode: "no-cors",
        cache: "no-store"
    });
    statusEl.textContent = "Подключено";
    statusEl.style.color = "#006400";
    alertBox.style.display = 'none'; // Скрываем алерт при успехе
} catch (e) {
    statusEl.textContent = "Нет подключения";
    statusEl.style.color = "#800000";
    alertBox.style.display = 'block'; // Показываем алерт при ошибке
}
}
document.getElementById('connectionAlert').addEventListener('click',
function() {
    this.style.display = 'none';
});

// Обработчики событий
statusEl.addEventListener('click', () => {
    ipInput.value = controllerURL;
    ipModal.style.display = 'block';
});

closeModal.addEventListener('click', () => {
    ipModal.style.display = 'none';
});

saveIpBtn.addEventListener('click', async () => {
    const newIP = ipInput.value.trim();
    if (newIP) {
        try {
            const response = await fetch('/update_ip', {
                method: 'POST',
                headers: {
                    'Content-Type': 'application/x-www-form-urlencoded',
                },
                body: `ip=${encodeURIComponent(newIP)}`
            });
            if (response.ok) {
                controllerURL = newIP;
                localStorage.setItem('controllerURL', newIP);
                ipModal.style.display = 'none';
                await checkConnection();
            }
        } catch (e) {
            console.error('Error updating IP:', e);
        }
    }
}
}

```

```

    });

    window.addEventListener('click', (event) => {
        if (event.target == ipModal) {
            ipModal.style.display = 'none';
        }
    });

    // Инициализация
    loadCurrentIP().then(() => {
        checkConnection();
        setInterval(checkConnection, 30000);
    });
    // Инициализация цветового круга
    const pickr = Pickr.create({
        el: '#colorPicker',
        theme: 'classic',
        default: localStorage.getItem('ledColor') || '#FFFFFF',
        components: {
            preview: true,
            opacity: false,
            hue: true,
            interaction: {
                hex: true,
                input: true,
            }
        }
    });

    const colorPreview = document.getElementById('colorPreview');
    const applyColorBtn = document.getElementById('applyColorBtn');

    // Установка начального цвета
    const initialColor = localStorage.getItem('ledColor') || '#FFFFFF';
    colorPreview.style.backgroundColor = initialColor;

    // При изменении цвета
    pickr.on('change', (color) => {
        const hexColor = color.toHEXA().toString();
        pickr.setColor(hexColor);
    });

    // Применение цвета
    applyColorBtn.addEventListener('click', async () => {
        const hexColor = pickr.getColor().toHEXA().toString();

        const rgb = hexToRgb(hexColor);
        if (rgb) {
            const response = await
fetch(`/color?r=${rgb.r}&g=${rgb.g}&b=${rgb.b}`);
            if (response.ok) {

```

```

        showSuccessMessage('Цвет применен!')
        colorPreview.style.backgroundColor = hexColor;
        localStorage.setItem('ledColor', hexColor);
    };
}
});

// Функция конвертации HEX в RGB
function hexToRgb(hex) {
    const result = /^#?([a-f\d]{2})([a-f\d]{2})([a-f\d]{2})$/i.exec(hex);
    return result ? {
        r: parseInt(result[1], 16),
        g: parseInt(result[2], 16),
        b: parseInt(result[3], 16)
    } : null;
}

// Функция показа уведомления
function showSuccessMessage(text) {
    const msg = document.createElement('div');
    msg.className = 'success-message';
    msg.textContent = text;
    document.body.appendChild(msg);

    setTimeout(() => {
        msg.classList.add('fade-out');
        setTimeout(() => msg.remove(), 500);
    }, 2000);
}

const modeButtons = document.querySelectorAll('.mode-btn');
const currentModeText = document.getElementById('currentModeText');

// Загружаем сохраненный режим (по умолчанию 0 - Статичный)
let currentMode = localStorage.getItem('ledMode') || '0';

// Обновляем интерфейс
function updateModeUI() {
    modeButtons.forEach(btn => {
        const mode = btn.dataset.mode;
        btn.classList.toggle('active', mode === currentMode);
        btn.disabled = mode === currentMode;
    });
}

// Обновляем текст текущего режима
const modeNames = {
    '0': 'Статичный',
    '1': 'Дыхание',
    '2': 'Радуга',
    '3': 'Музыка'
};
};

```

```

        currentModeText.textContent = modeNames[currentMode];
    }

    // Обработчики кликов
    modeButtons.forEach(btn => {
        btn.addEventListener('click', async function() {
            const selectedMode = this.dataset.mode;

            // Отправляем на сервер
            const formData = new FormData();
            formData.append('mode', selectedMode);

            const response = await fetch('/mode', {
                method: 'POST',
                body: formData
            });
            if (response.ok) {
                currentMode = selectedMode;
                localStorage.setItem('ledMode', currentMode);
                updateModeUI();
                showSuccessMessage('Режим изменен!');}
            });
        });

    // Инициализация
    updateModeUI();

    });
</script>
</body>
</html>

```

1.4 Аппаратная часть проекта

Код на C для загрузки на контроллер ESP8266 через Arduino IDE представлен в листинге 4.

Листинг 4 – Код для контроллера

```

#include <ESP8266WiFi.h>
#include <ESP8266WebServer.h>
#include <ArduinoJson.h>
#include <math.h>

// ——— Настройки Wi-Fi ———
const char* WIFI_SSID      = "Redmi Note 12";
const char* WIFI_PASSWORD = "sarik323";

// ——— Пины ленты и микрофона ———
#define PIN_R    D5

```

```

#define PIN_G    D3
#define PIN_B    D2
#define PIN_MIC  A0

// — Глобальные переменные состояния —————
ESP8266WebServer server(80);

bool  isOn = false;
uint8_t r    = 255;
uint8_t g    = 255;
uint8_t b    = 255;
uint8_t mode = 0; // 0=статичный, 1=дыхание, 2=радуга, 3=музыка

// Удалили калибровочные переменные
const int noiseLevel = 62; // Базовый уровень шума (тишина)
const int maxSignal = 400 - noiseLevel; // Максимальный полезный сигнал

void setup() {
    Serial.begin(9600);

    // Инициализация пинов
    pinMode(PIN_R, OUTPUT);
    pinMode(PIN_G, OUTPUT);
    pinMode(PIN_B, OUTPUT);
    analogWrite(PIN_R, 0);
    analogWrite(PIN_G, 0);
    analogWrite(PIN_B, 0);

    // Подключение к Wi-Fi
    WiFi.mode(WIFI_STA);
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    Serial.println();
    Serial.print("Ready, IP = ");
    Serial.println(WiFi.localIP());

    // Маршруты веб-сервера
    server.on("/", handleRoot);
    server.on("/on", handleOn);
    server.on("/off", handleOff);
    server.on("/color", handleColor);
    server.on("/mode", handleModeSetting);
    server.begin();
}

void loop() {
    server.handleClient();
    handleModeAnimations();
}

```

```

}

// ——— Обработчики веб-запросов ———
void handleRoot() {
    String html = "<!DOCTYPE html><html>"
                "<head><title>LED Control</title></head>"
                "<body>"
                "<h1>LED Strip Control</h1>"
                "<p><a href='/on'><button>Включить</button></a> "
                "<a href='/off'><button>Выключить</button></a></p>"
                "<form action='/color' method='GET'>"
                "R: <input type='number' name='r' min=0 max=255 value='" +
String(r) + "'><br>"
                "G: <input type='number' name='g' min=0 max=255 value='" +
String(g) + "'><br>"
                "B: <input type='number' name='b' min=0 max=255 value='" +
String(b) + "'><br>"
                "<input type='submit' value='Установить цвет'>"
                "</form>"
                "<form action='/mode' method='GET'>"
                "Режим: <select name='m'>"
                "<option value='0'>Статичный</option>"
                "<option value='1'>Дыхание</option>"
                "<option value='2'>Радуга</option>"
                "<option value='3'>Музыка</option>"
                "</select>"
                "<input type='submit' value='Выбрать режим'>"
                "</form>"
                "</body></html>";
    server.send(200, "text/html", html);
}

void handleOn() {
    isOn = true;
    applyColor(r, g, b);
    server.send(200, "application/json", "{\"status\":\"on\"}");
}

void handleOff() {
    isOn = false;
    applyColor(0, 0, 0);
    server.send(200, "application/json", "{\"status\":\"off\"}");
}

void handleColor() {
    if (server.hasArg("r")) r = server.arg("r").toInt();
    if (server.hasArg("g")) g = server.arg("g").toInt();
    if (server.hasArg("b")) b = server.arg("b").toInt();

    r = constrain(r, 0, 255);
    g = constrain(g, 0, 255);

```

```

b = constrain(b, 0, 255);

if(isOn && mode == 0) {
    applyColor(r, g, b);
}

server.send(200, "application/json",
    "{ \"r\": " + String(r) +
    ", \"g\": " + String(g) +
    ", \"b\": " + String(b) + " }");
}

void handleModeSetting() {
    if (server.hasArg("m")) {
        mode = server.arg("m").toInt();
        if(mode < 0 || mode > 3) mode = 0;
        Serial.print("Режим изменен на: "); Serial.println(mode); // Отладка
    }
    server.send(200, "application/json", "{ \"mode\": " + String(mode) + " }");
}

// — Управление лентой —
void applyColor(uint8_t R, uint8_t G, uint8_t B) {
    analogWrite(PIN_R, 255 - R);
    analogWrite(PIN_G, 255 - G);
    analogWrite(PIN_B, 255 - B);
}

void handleModeAnimations() {
    if(!isOn) return;

    static unsigned long lastUpdate = 0;
    static uint8_t rainbowPhase = 0;

    switch(mode) {
        case 1: // Дыхание
            if(millis() - lastUpdate > 30) {
                float brightness = (sin(millis()/1000.0 * PI) + 1) / 2;
                applyColor(r * brightness, g * brightness, b * brightness);
                lastUpdate = millis();
            }
            break;

        case 2: // Радуга
            if(millis() - lastUpdate > 20) {
                rainbowPhase = (rainbowPhase + 1) % 255;
                applyColor(
                    wheel((rainbowPhase) % 255),
                    wheel((rainbowPhase + 85) % 255),
                    wheel((rainbowPhase + 170) % 255)
                );
            }
    }
}

```

```

        lastUpdate = millis();
    }
    break;

case 3: // Музыка
{
    static unsigned long prevMusicUpdate = 0;
    if(millis() - prevMusicUpdate < 10) break;
    prevMusicUpdate = millis();

    int micValue = analogRead(PIN_MIC);
    int signal = constrain(micValue - noiseLevel, 0, maxSignal);

    float logLevel = log10(signal + 1) / log10(maxSignal + 1);
    float dynamicLevel = pow(logLevel, 2.5);

    static float smoothedLevel = 0;
    smoothedLevel = 0.3 * dynamicLevel + 0.7 * smoothedLevel;

    int level = constrain(smoothedLevel * 255, 0, 255);
    applyColor(r * level / 255, g * level / 255, b * level / 255);

}
break;
}
}

// Вспомогательная функция для радуги
uint32_t wheel(byte pos) {
    pos = 255 - pos;
    if(pos < 85) return (255 - pos * 3) << 16 | 0 << 8 | pos * 3;
    if(pos < 170) { pos -= 85; return 0 << 16 | pos * 3 << 8 | (255 - pos * 3); }
    pos -= 170; return pos * 3 << 16 | (255 - pos * 3) << 8 | 0;
}

```

1.5 Сборка документации проекта

Соберем отчет по проекту с помощью nbconvert. Для этого разобьем app.py на блоки в report.ipynb и конвертируем командой `jupyter nbconvert --to pdf report.ipynb`. Полученная документация представлена на рис. 3.


```

In [ ]: import socket
from flask import Flask, render_template, request, redirect, jsonify, session
import requests
import os

app = Flask(__name__)
app.secret_key = os.urandom(24)

In [ ]: def get_local_ip():
    """
    Определяет локальный IP-адрес машины в сети посредством открытия
    временного UDP-сокета к публичному DNS Google.
    Возвращает строку вида '192.168.x.x' или '127.0.0.1' при ошибке.
    """
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        s.connect(("8.8.8.8", 80))
        local_ip = s.getsockname()[0]
        s.close()
        return local_ip
    except Exception:
        return "127.0.0.1"

DEFAULT_ESP_IP = 'http://192.168.1.100'

def get_esp_ip():
    """
    Возвращает текущий IP ESP-контроллера из сессии Flask.
    Если в сессии нет значения, возвращает DEFAULT_ESP_IP.
    """
    return session.get('esp_ip', DEFAULT_ESP_IP)

In [ ]: @app.route('/')
def index():
    """
    Главная страница: отображает шаблон control_panel с текущим IP контроллера.
    """
    return render_template('index.html', current_ip=get_esp_ip())

In [ ]: @app.route('/update_ip', methods=['POST'])
def update_ip():
    """
    Обработывает POST-запрос для обновления IP контроллера.
    Проверяет, что строка начинается с http:// или https://
    и сохраняет её в сессии.
    """
    new_ip = request.form.get('ip')
    if new_ip and new_ip.startswith(('http://', 'https://')):
        session['esp_ip'] = new_ip
        return jsonify({'status': 'success', 'ip': new_ip})
    return jsonify({'status': 'error'}), 400

@app.route('/power/cstate')
def power_control(state):
    """

```

```

Включает или выключает LED-ленту.
При state=="on" отправляет запрос на /on, иначе /off.
После запроса перенаправляет на '/' методом HTTP 302.
"""
esp_ip = get_esp_ip()
if state == 'on':
    requests.get(f'{esp_ip}/on')
else:
    requests.get(f'{esp_ip}/off')
return redirect('/')

@app.route('/color')
def color():
    """
    Устанавливает цвет: принимает параметры r, g, b в query-string.
    Перенаправляет обратно на главную.
    """
    esp_ip = get_esp_ip()
    r = request.args.get('r')
    g = request.args.get('g')
    b = request.args.get('b')
    if all([r, g, b]):
        requests.get(f'{esp_ip}/color?r={r}&g={g}&b={b}')
    return redirect('/')

@app.route('/mode', methods=['POST'])
def set_mode():
    """
    Изменяет режим работы ленты: статичный, дыхание, радуга или музыка.
    Получает параметр 'mode' из формы и отправляет запрос к ESP.
    """
    esp_ip = get_esp_ip()
    mode = request.form.get('mode', '0')
    requests.get(f'{esp_ip}/mode?m={mode}')
    return ('', 200)

In [ ]: if __name__ == '__main__':
    """
    Запускает Flask-сервер на локальном IP и порту 5050.
    """
    app.run(host=get_local_ip(), port=5050)

```

Рисунок 3 – Документация проекта